

সফটওয়্যার প্রয়োজনীয়তা বিবরণী · SOFTWARE REQUIREMENTS SPECIFICATION

Recipe Food ERP

সম্পূর্ণ সিস্টেম বিশ্লেষণ — মডিউল, ডেটা মডেল, গণনা-সূত্র ও আন্তঃসংযোগ

কোড-ভিত্তিক টেকনিক্যাল বিশ্লেষণ (Reverse-Engineered SRS)

প্রজেক্ট: erp-recipeFood (Next.js + TypeScript + Firebase Realtime Database)

ডকুমেন্টের ধরন: সম্পূর্ণ কোডবেস বিশ্লেষণ করে তৈরি বিস্তারিত SRS — প্রতিটি মডিউলের ব্যবসায়িক নিয়ম, গণনা-সূত্র এবং এনটিটিগুলির মধ্যে সংযোগ

তৈরির তারিখ: ১২ সেপ্টেম্বর, ২০২৬

সূচিপত্র

ভূমিকা

উদ্দেশ্য

পরিধি

প্রযুক্তি স্ট্যাক ও ডেটা সংরক্ষণ পদ্ধতি

মূল ডিজাইন নীতি — "Derive, Don't Store"

ব্যবহারকারী, রোল ও পারমিশন

ডেটা মডেল — এনটিটি ও সম্পর্ক

এনটিটি ডোমেইনভিত্তিক তালিকা

মূল রিলেশনশিপ ডায়াগ্রাম

পণ্য, রেসিপি ও প্রাইসিং ক্যাটালগ

ভিলার, ডিপো ও ভিলার ক্যাটাগরি

ক্রয় ও ভেন্ডর ব্যবস্থাপনা (Purchase Module)

উৎপাদন — কাঁচামাল থেকে ফিনিশড গুডস (Production)

স্টক নিয়ন্ত্রণ (Stock Adjustment · Stock Count · QC Hold)

রেট কার্ড / ইনভয়েসিং — মূল বিক্রয় ইঞ্জিন

ডিসকাউন্ট প্রোডাক্ট ও কমিশন-চেইন প্রাইসিং

প্রোডাক্ট রিটার্ন

সেলস অর্ডার মডিউল (বর্তমানে নিষ্ক্রিয়)

সেলস রিপোর্ট ও কোম্পানি আনিংস

এক্সপেন্স ও বাজেট ব্যবস্থাপনা

লোন ম্যানেজমেন্ট

ক্যাশ মেইনটেন্যান্স ও ইনভেস্টর

অ্যাকাউন্টিং — চার্ট অফ অ্যাকাউন্টস, জার্নাল, ব্যাংক, লেজার, ব্যালান্স শিট

কমিশন রুল ও সেলস টার্গেট (শুধু কাঠামো, বাস্তবায়িত নয়)

ড্যাশবোর্ড ও রিপোর্টস হাব

আন্তঃমডিউল ডেটা-ফ্লো (সম্পূর্ণ ছবি)

গণনা-সূত্রের মাস্টার রেফারেন্স টেবিল

সিস্টেমের সীমাবদ্ধতা ও গ্যাপ

অ-কার্যকরী প্রয়োজনীয়তা

১. ভূমিকা

১.১ উদ্দেশ্য

এই ডকুমেন্টটি **Recipe Food ERP** সিস্টেমের সম্পূর্ণ কোডবেস বিশ্লেষণ করে তৈরি একটি বিস্তারিত সফটওয়্যার প্রয়োজনীয়তা বিবরণী (SRS)। এখানে সিস্টেমের প্রতিটি মডিউলের ব্যবসায়িক নিয়ম, প্রতিটি অঙ্কের প্রকৃত সূত্র (formula), এবং কোন এনটিটি/মডিউল কার সাথে কীভাবে যুক্ত — তা কোডের প্রকৃত লজিক (provider.tsx, utils.ts, types.ts এবং প্রতিটি অ্যাডমিন স্ক্রিন) থেকে হুবহু তুলে ধরা হয়েছে। উদ্দেশ্য হলো — ব্যবস্থাপনা, ডেভেলপার এবং অডিটর — সবাই যেন একটি একক দলিল থেকে বুঝতে পারেন সিস্টেমটি প্রকৃতপক্ষে কীভাবে কাজ করে।

১.২ পরিধি

Recipe Food ERP একটি খাদ্য উৎপাদন ও পরিবেশন ব্যবসার জন্য তৈরি সম্পূর্ণ ব্যবস্থাপনা সিস্টেম, যা নিম্নলিখিত কার্যক্রম কভার করে:

- কাঁচামাল ও প্যাকেজিং মালামাল ক্রয় (Purchase) ও ভেন্টর ব্যবস্থাপনা
- কাঁচামাল থেকে ফিনিশড গুডস উৎপাদন (Production)
- স্টক নিয়ন্ত্রণ, ফিজিক্যাল কাউন্ট, কোয়ালিটি কন্ট্রোল হোল্ড
- ডিলার/ডিপো ভিত্তিক ইনভয়েসিং ও প্রাইসিং (Rate Card)
- প্রোডাক্ট রিটার্ন ব্যবস্থাপনা
- এক্সপেন্স, বাজেট, লোন, ক্যাশ মেইনটেন্যান্স, ইনভেন্টরি ইকুইটি
- সম্পূর্ণ ডাবল-এন্ট্রি অ্যাকাউন্টিং — চার্ট অফ অ্যাকাউন্টস, জার্নাল এন্ট্রি, ব্যাংক লেনদেন, জেনারেল লেজার, ট্রায়াল ব্যালান্স, ব্যালান্স শিট
- সেলস রিপোর্ট, কোম্পানি আর্নিংস, ড্যাশবোর্ড ও রিপোর্টস হাব
- ইউজার/রোল/পারমিশন ব্যবস্থাপনা

১.৩ প্রযুক্তি স্ট্যাক ও ডেটা সংরক্ষণ পদ্ধতি

ফ্রন্টএন্ড	Next.js (App Router) + React + TypeScript, Tailwind-ভিত্তিক UI কম্পোনেন্ট
ডেটাবেজ	Firebase Realtime Database (Firestore নয়) — <code>erp/...</code> path-এর নিচে একটিমাত্র বড় JSON ট্রি হিসেবে সব ডেটা রাখা হয়
লেখার পদ্ধতি	প্রতিটি বিজনেস অপারেশন একটিমাত্র <code>update(ref(db,'erp'), {...})</code> মাল্টি-পাথ ব্যাচ কলে সম্পন্ন হয় — একসাথে একাধিক এনটিটি আপডেট হয়
স্টেট ম্যানেজমেন্ট	<code>src/lib/erp/provider.tsx</code> — একটি বড় React Context, যা সব ব্যবসায়িক লজিক ও Firebase read/write ধারণ করে
গণনার ভিত্তি	<code>src/lib/erp/utils.ts</code> — সকল রিপোর্ট/ব্যালান্স/টোটাল গণনার pure function সমূহ
টাইপ/স্কিমা	<code>src/lib/erp/types.ts</code> — প্রতিটি এনটিটির গঠন ও তার ইনপুট টাইপ (১৭৫৮ লাইন)

গুরুত্বপূর্ণ টেকনিক্যাল নোট: Firebase Realtime Database-এ কোনো সার্ভার-সাইড ট্রানজ্যাকশন লক নেই। প্রতিটি গণনা ক্লায়েন্ট (ব্রাউজার)-এ, শেষবার পড়া ডেটার স্ল্যাপশটের ভিত্তিতে করা হয় এবং তারপর লেখা হয়। একাধিক ব্যবহারকারী একই সময়ে একই রেকর্ড এডিট করলে race condition-এর সম্ভাবনা তাত্ত্বিকভাবে থেকে যায়।

১.৪ মূল ডিজাইন নীতি — "Derive, Don't Store" (গণনা করো, সংরক্ষণ করো না)

পুরো সিস্টেম জুড়ে একটি সুস্পষ্ট এবং সামঞ্জস্যপূর্ণ স্থাপত্য-নীতি অনুসরণ করা হয়েছে — **কোনো ব্যালান্স/টোটাল ফিল্ড সরাসরি সংরক্ষণ (cache) করা হয় না**; বরং প্রতিটি স্ক্রিন প্রতিবার রেন্ডার হওয়ার সময় মূল লেনদেন-রেকর্ড থেকে লাইভ গণনা করে। এর ফলে ডেটা কখনো "stale" (পুরনো/ভুল) হয়ে যাওয়ার সুযোগ থাকে না। যেসব মান এভাবে সবসময় লাইভ গণনা হয়:

যে মান	কোথা থেকে গণনা হয়	ফাংশন
ডিলারের বকেয়া (Due)	ঐ ডিলারের অ-বাজিল <code>OrderRecord</code> সমূহের <code>due</code> -এর যোগফল	<code>computeDealerDue</code>
ভেন্টরের বকেয়া (Payable)	ঐ ভেন্টরের সকল <code>PurchaseRecord.due</code> -এর যোগফল	<code>computeVendorDue</code>
লোন ব্যালেন্স	সকল <code>LoanTransactionRecord</code> (withdrawal - repayment)	<code>computeLoanBalance</code>
বাজেটের প্রকৃত খরচ (Actual)	মিলে যাওয়া ক্যাটাগরি ও পিরিয়ডের <code>ExpenseRecord</code> সমূহ	<code>getBudgetActual</code>
চার্ট অফ অ্যাকাউন্টসের ব্যালেন্স	<code>openingBalance + Σ(debit - credit)</code> — <code>ledgerEntries</code> থেকে	<code>buildGeneralLedger</code>
ট্রায়াল ব্যালান্স, ব্যালান্স শিট	জেনারেল লেজারের উপর ভিত্তি করে	<code>buildTrialBalance</code> , <code>buildBalanceSheet</code>

এই ডকুমেন্টের প্রতিটি অধ্যায়ে যেখানেই "কখনো সংরক্ষিত হয় না, সবসময় লাইভ গণনা হয়" উল্লেখ থাকবে, তা এই একই নীতির প্রয়োগ।

২. ব্যবহারকারী, রোল ও পারমিশন

সিস্টেমে একটি রোল-ভিত্তিক অ্যাক্সেস কন্ট্রোল (RBAC) কাঠামো আছে।

এনটিটি	বিবরণ
PermissionDefinition	একটি নির্দিষ্ট <code>module:action</code> পারমিশনের ক্যাটালগ এন্ট্রি (যেমন <code>orders:approve</code> , <code>finance:edit</code> , <code>products:create</code>)
RoleRecord	<code>id</code> , <code>name</code> , <code>permissions</code> — একটি রোলের অধীনে কোন কোন পারমিশন সক্রিয় তার সেট
UserRecord	<code>id</code> , <code>name</code> , <code>loginId</code> , <code>email</code> , <code>phone</code> , <code>roleId</code> → <code>RoleRecord</code> , <code>status</code>
LoginHistoryRecord	লগইন করা প্রতিটি সেশনের IP/ডিভাইস লগ
ActivityRecord	প্রতিটি গুরুত্বপূর্ণ অপারেশনের অডিট-ট্রেইল — কে, কী, কখন, পুরনো মান বনাম নতুন মান
NotificationRecord	info/warning/critical লেভেলের বার্তা, নির্দিষ্ট রোলের উদ্দেশ্যে পাঠানো (যেমন স্টক কম, বাজেট ওভাররান)

```
hasPermission(data, user, permission)
  = data.roles[user.roleId]?.permissions?.[permission] === true
```

UI-এর প্রতিটি সংবেদনশীল বাটন/অ্যাকশন (যেমন এক্সপেন্স অনুমোদন, প্রোডাক্ট ডিলিট, ফাইন্যান্স এডিট) এই `hasPermission()` চেকের মাধ্যমে দেখানো/লুকানো/সক্রিয়-নিষ্ক্রিয় হয়। `writeActivity()` ফাংশনটি প্রতিটি সেভ/ডিলিট/অ্যাপ্রুভ অপারেশনের পরে একটি অডিট এন্ট্রি লেখে, এবং `writeNotification()` নির্দিষ্ট রোলের ব্যবহারকারীদের সতর্কবার্তা পাঠায়।

৩. ডেটা মডেল — এনটিটি ও সম্পর্ক

৩.১ এনটিটি ডোমেইনভিত্তিক তালিকা

নিচে `src/lib/erp/types.ts`-এ সংজ্ঞায়িত সকল এনটিটি ডোমেইন অনুযায়ী সাজানো হলো। প্রতিটি এনটিটি Firebase-এ `erp/<collectionName>/<id>` পাথে `Record<string, XRecord>` (id দিয়ে ম্যাপ করা) আকারে সংরক্ষিত থাকে।

ক) ইউজার/রোল/নিরাপত্তা

`PermissionDefinition`, `RoleRecord`, `UserRecord`, `LoginHistoryRecord`, `ActivityRecord`, `NotificationRecord`

খ) বিক্রয়-চ্যানেল পার্টি

`DealerRecord` (name, proprietorName, address, phone, categoryId→DealerCategory, depotId→Depot), `DealerCategoryRecord` (নামভিত্তিক শ্রেণিবিন্যাস — Wholesaler/SR/Trade Sales/Commission ইত্যাদি), `DepotRecord`

গ) পণ্য ও ক্যাটালগ

`ProductRecord` (মূল পণ্য তালিকা — rawRate, manufRate, depotRate, dealerRate, tpRate, mrpRate, stockQty, minStock, maxStock, status), `DiscountProductRecord` (কমিশন-চেইন প্রাইসিং ক্যাটালগ), `BatchRecord` (FEFO ব্যাচ ট্র্যাকিং)

ঘ) বিক্রয় / ইনভয়েসিং

`OrderRecord` (নিষ্ক্রিয় মডিউল — অধ্যায় ১২ দ্রষ্টব্য), `SalesReturnRecord`, `CollectionRecord`, `RateCardRecord` (মূল ইনভয়েস — অধ্যায় ৯), `ProductReturnRecord` (অধ্যায় ১১)

ঙ) ক্রয় ও ভেন্ডর

`VendorRecord`, `PurchaseMaterialRecord`, `PurchaseRecord`, `VendorPaymentRecord`, `MaterialUsageRecord`

চ) উৎপাদন

`FinishedGoodsRecord`, `ProductionBatchRecord`, `StockShortfallRecord`

ছ) কোয়ালিটি কন্ট্রোল ও স্টক নিয়ন্ত্রণ

`QualityCheckRecord`, `QcHoldRecord`, `StockAdjustmentRecord`, `StockCountRecord`

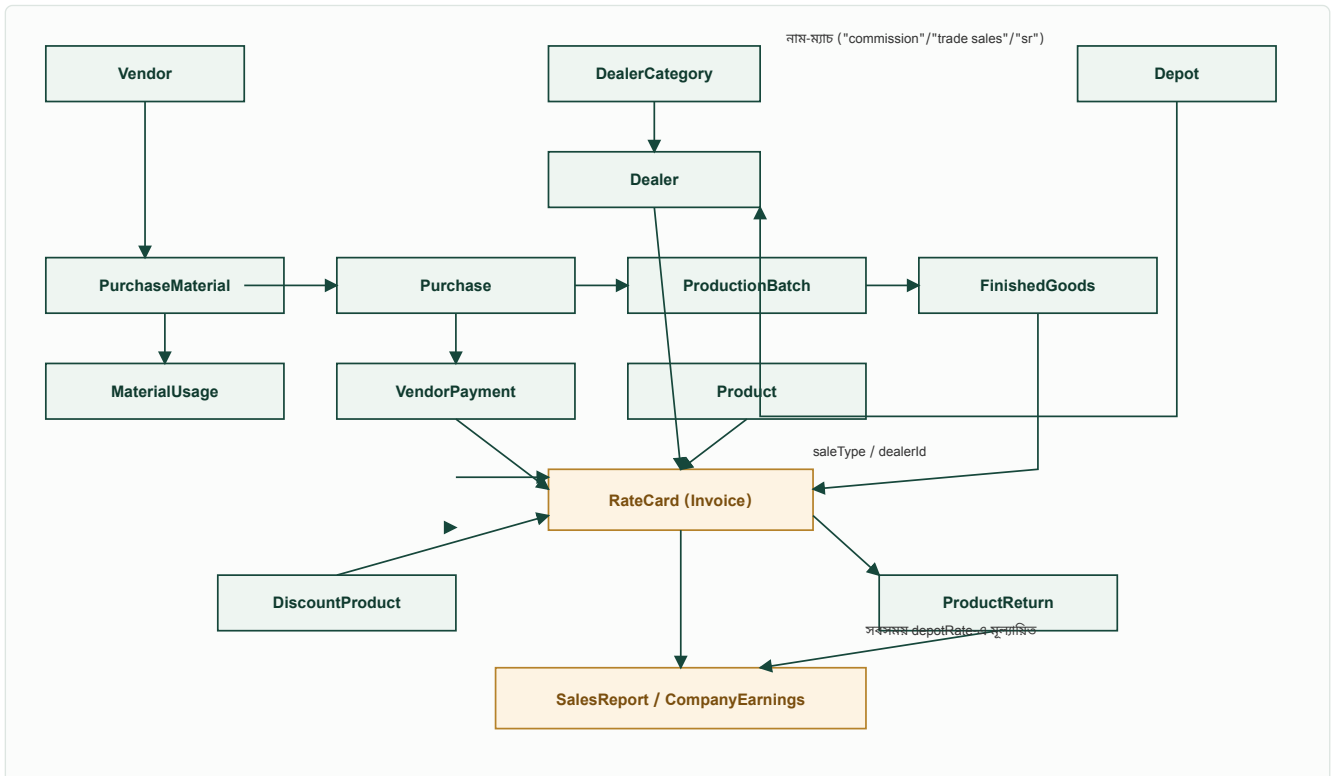
জ) ফাইন্যান্স / অ্যাকাউন্টিং

`LedgerEntryRecord`, `ChartOfAccountRecord`, `JournalEntryRecord`, `BankAccountRecord`, `BankTransactionRecord`, `ExpenseRecord`, `LoanAccountRecord`, `LoanTransactionRecord`

ঝ) রিপোর্টিং / টার্গেট / কমিশন

`SalesTargetRecord`, `CommissionRuleRecord`, `CommissionPayoutRecord`

৩.২ মূল রিলেশনশিপ ডায়াগ্রাম



উপরের ডায়াগ্রামটি প্রধান তথ্য-প্রবাহ দেখায়: ভেন্ডর → ক্রয় → কাঁচামাল স্টক → উৎপাদন → ফিনিশড গুডস/প্রোডাক্ট → রেট কার্ড (ইনভয়েস) → ডিলার → সেলস রিপোর্ট। সম্পূর্ণ ফিল্ড-লেভেল সম্পর্ক প্রতিটি নির্দিষ্ট অধ্যায়ে বিস্তারিত দেওয়া আছে।

৪. পণ্য, রেসিপি ও প্রাইসিং ক্যাটালগ

৪.১ ProductRecord — মূল পণ্য তালিকা

প্রতিটি পণ্যের জন্য ছয়টি রোট-কলাম সংরক্ষিত থাকে, যা রোট কার্ড তৈরির সময় প্রাথমিক মান হিসেবে ব্যবহৃত হয়: `rawRate`, `manufRate`, `depotRate`, `dealerRate`, `tpRate`, `mrpRate`। স্টক-স্ট্যাটাস স্বয়ংক্রিয়ভাবে গণনা হয়:

```
getProductStatus(stockQty, minStock):
  stockQty <= 0      → 'out-of-stock'
  stockQty <= minStock → 'low-stock'
  অন্যথায়           → 'active'
```

SKU স্বয়ংক্রিয়ভাবে তৈরি হয় (`generateSku`, `StockOverviewScreen.tsx`) — নাম থেকে বড়হাতের অক্ষর নিয়ে, অ-আক্ষরিক চিহ্ন `-` দিয়ে প্রতিস্থাপন করে, ২০ অক্ষরে ছেঁটে, এবং কোনো সংঘর্ষ হলে সংখ্যা যোগ করে ইউনিক করা হয়।

৪.২ DiscountProductRecord — কমিশন-চেইন প্রাইসিং ক্যাটালগ

এটি একটি স্বতন্ত্র সমান্তরাল ক্যাটালগ, যা শুধুমাত্র তখনই ব্যবহৃত হয় যখন রোট কার্ডের Sale Type "commission" নামযুক্ত ডিলার ক্যাটাগরি হয় (বিস্তারিত অধ্যায় ১০-এ)। এতে `srCommissionPercent` (ডিফল্ট ৮%) এবং `tpPercent` সংরক্ষিত থাকে, কিন্তু ডেরাইভড রোট (`srRate`, `tpRate`) সংরক্ষিত হয় না — প্রতিবার লাইভ গণনা করা হয়:

```
computedDiscountProductRates(dealerRate, srCommissionPercent, tpPercent):
  srRate = dealerRate * (1 + srCommissionPercent / 100)
  tpRate = srRate * (1 + tpPercent / 100)
```

৪.৩ FinishedGoodsRecord — উৎপাদিত পণ্যের ক্যাটালগ

শুধুমাত্র প্রোডাকশন ব্যাচ থেকে আসা পণ্যের জন্য আলাদা ক্যাটালগ (মূল Product তালিকা থেকে সম্পূর্ণ পৃথক)। এর ছয়টি রোট-কলাম **হাতে করে এন্ট্রি করা**, প্রকৃত কাঁচামাল-ক্রয়-মূল্যের সাথে কোনো স্বয়ংক্রিয় সংযোগ নেই (অধ্যায় ৭-এ বিস্তারিত)।

৪.৪ BatchRecord — FEFO ব্যাচ ট্র্যাকিং

Expiry-ভিত্তিক ব্যাচ ট্র্যাকিং, শুধুমাত্র নিষ্ক্রিয় Sales Order মডিউল (অধ্যায় ১২) ব্যবহার করে। স্টক কন্ট্রোল স্ক্রিনে "days left" ব্যাজ দেখানোর জন্য ব্যবহৃত হয়: `daysUntil = ceil((expiryDate - এখন) / 86400000)`।

৫. ডিলার, ডিপো ও ডিলার ক্যাটাগরি

Company → Depot → Dealer — এই তিন-স্তরের বিতরণ কাঠামো সিস্টেমের কেন্দ্রীয় ধারণা।

এনটিটি	মূল ফিল্ড	সম্পর্ক
DealerRecord	name, proprietorName, address, phone	categoryId → DealerCategory , depotId → Depot
DealerCategoryRecord	name	—
DepotRecord	name, proprietorName, address, phone	—

কোনো ডিলার/ডেপোর ক্রেডিট-লিমিট বা বকেয়া ফিল্ড সরাসরি সংরক্ষিত হয় না — বকেয়া সবসময় computeDealerDue / computeVendorDue দিয়ে লাইভ গণনা করা হয় (অধ্যায় ১.৪ দ্রষ্টব্য)।

৫.১ ডিলার ক্যাটাগরির নাম কীভাবে প্রাইসিং লজিক নিয়ন্ত্রণ করে

এটি সিস্টেমের অন্যতম গুরুত্বপূর্ণ এবং সুক্ষ্ম নিয়ম — একটি ক্যাটাগরির id নয়, বরং তার নামের টেক্সট (কেস-ইনসেনসিটিভ সাবস্ট্রিং মিল) ব্যবহার করে বিভিন্ন আচরণ নির্ধারিত হয়:

হেল্পার ফাংশন	শর্ত	প্রভাব
isCommissionSaleType	নামে "commission" শব্দ থাকলে	রেট কার্ড লাইনের প্রোডাক্ট-পিকার সাধারণ Product List-এর বদলে Discount Product List দেখায়
isTradeSalesType	নামে "trade sales" শব্দ থাকলে	ইনভয়েস প্রিন্ট মেনুতে অতিরিক্ত "Retail Sales" ভাউচার অপশন যুক্ত হয়
isSrDistributorType	নাম /\bsr\b/i প্যাটার্নে মিলে গেলে	প্রোডাক্ট রিটার্ন স্ক্রিনে ডিলারদের "SR" গ্রুপে শ্রেণিবদ্ধ করে

ঝুঁকি: deleteDealerCategory -এ কোনো রেফারেন্স-চেক নেই — অন্য কোথাও (Dealer.categoryId, RateCard.saleType) ব্যবহৃত একটি ক্যাটাগরি ডিলিট করা গেলে সেই রেফারেন্সগুলো "Unclassified" (অশ্রেণীবদ্ধ) হিসেবে দেখাবে।

৬. ক্রয় ও ভেন্ডর ব্যবস্থাপনা (Purchase Module)

৬.১ PurchaseMaterialRecord — কাঁচামাল/প্যাকেজিং ক্যাটালগ

প্রতিটি উপাদানের `category` (raw_material/packaging_material), `unit` (kg/pcs), `stockQty`, `minStock` সংরক্ষিত থাকে। প্যাকেজিং মালামালের জন্য "পিস" রূপান্তর সূত্র:

```
computeMaterialAvailablePieces(material):  
    unit === 'kg' && unitWeightGrams > 0 → floor((stockQty * 1000) / unitWeightGrams)  
    unit === 'pcs' && capacityPerUnit > 0 → floor(stockQty * capacityPerUnit)
```

গ্যাপ: `PurchaseMaterialRecord` -এ কোনো cost/average-cost ফিল্ড নেই। ফলে সিস্টেমে কাঁচামালের "ইনভেন্টরি ভ্যালু" নামে কোনো ধারণা নেই।

৬.২ createPurchase — একটি ক্রয় রেকর্ড করা

```
প্রতি লাইন: amount = qty * rate (qty > 0, rate >= 0 আবশ্যক)  
totalAmount = Σ line.amount  
paid = clamp(input.paid, 0, totalAmount)  
due = totalAmount - paid  
স্টক: purchaseMaterials[materialId].stockQty += qty (একই মালামাল একাধিক লাইনে থাকলে যোগফল প্রয়োগ হয়)
```

ক্যাশ পোস্টিং (`buildPurchaseCashEntries`) — পরিশোধিত টাকা (`paid`) কাঁচামাল বনাম প্যাকেজিং লাইনের মূল্য-অনুপাতে ভাগ করে `CashMaintenanceRecord` -এ পোস্ট হয়:

```
rawShare = rawTotal / (rawTotal + packagingTotal) (মোট শূন্য হলে rawShare = 1)  
rawAmount = paid * rawShare  
packagingAmount = paid - rawAmount
```

গুরুত্বপূর্ণ গ্যাপ: ক্রয় (Purchase) এবং ভেন্ডর পেমেন্ট কখনো সাধারণ ডাবল-এন্ট্রি লেজারে (`ledgerEntries`) পোস্ট হয় না — শুধু আলাদা Cash Maintenance লগে যায়। ফলে Chart of Accounts-এর "Accounts Payable"/"Supplier" অ্যাকাউন্ট বাস্তব ক্রয়-কার্যক্রম থেকে কখনো স্বয়ংক্রিয়ভাবে আপডেট হয় না — শুধুমাত্র ম্যানুয়াল Journal Entry দিয়েই তা সম্ভব।

ভেন্ডরের বকেয়া সবসময় লাইভ গণনা: `computeVendorDue = Σ purchase.due` (ঐ ভেন্ডরের সকল ক্রয়ের বকেয়ার যোগফল)।

৬.৩ recordVendorPayment — ভেন্ডরকে বকেয়া পরিশোধ

```
amount > 0 এবং amount <= purchase.due (আবশ্যক)  
nextDue = purchase.due - amount  
nextPaid = purchase.paid + amount
```

একই `buildPurchaseCashEntries` রাত্তিও-স্প্লিট লজিক প্রযোজ্য হয় (মূল ক্রয়ের আইটেম-মিশ্রণ অনুসারে)। এই ফাংশনও কোনো স্টক বা লেজার প্রভাব ফেলে না।

৬.৪ createMaterialUsage — মালামাল ব্যবহার/ইস্যু

```
nextStock = material.stockQty - qty (ইচ্ছাকৃতভাবে ঋণাত্মক হতে দেওয়া হয়, যাতে ঘাটতি দৃশ্যমান হয়)
```

`nextStock <= material.minStock` হলে সংশ্লিষ্ট রোলদের (super_admin, manager, warehouse_manager, production_manager) কাছে লো-স্টক নোটিফিকেশন যায়।

৬.৫ মুছে ফেলার (Delete) নিয়ম

- `deletePurchaseMaterial` — ক্রয়/ব্যবহারের ইতিহাস থাকলে ব্লক হয়
- `deletePurchase` — স্টক পূর্ণ বিপরীতমুখী করে, লিংকড ক্যাশ এন্ট্রি ও সব `VendorPaymentRecord` ক্যাসকেড ডিলিট করে
- `deleteMaterialUsage` — `stockQty += usage.qty` ফিরিয়ে দেয়

৭. উৎপাদন — কাঁচামাল থেকে ফিনিশড গুডস (Production)

৭.১ createProductionBatch — মূল "রেসিপি" রূপান্তর যুক্তি

একটি ব্যাচে ঠিক একটি কাঁচামাল একাধিক ফিনিশড-গুডস প্যাক-সাইজে রূপান্তরিত হয়। এখানেই সিস্টেমের একমাত্র "রেসিপি/BOM" যুক্তি:

```
প্রতি আউটপুট লাইনে:  
rawKgConsumed      = qtyProduced * unitWeightKg  
rawKgConsumedTotal  = Σ rawKgConsumed (সব আউটপুট লাইনের যোগফল)  
nextRawStock        = rawMaterial.stockQty - rawKgConsumedTotal (ঋণাত্মক হতে পারে)  
finishedGoods.stockQty += qtyProduced (প্রতিটি আউটপুট লাইনের জন্য, একই FG একাধিকবার থাকলে যোগফল)
```

সবচেয়ে গুরুত্বপূর্ণ গ্যাপ: এখানে কোনো **খরচ (cost) গণনা হয় না** — এটি নিছক একটি ওজন/পরিমাণ রূপান্তর সূত্র। `FinishedGoodsRecord`-এর ছয়টি রেট কলাম (`rawRate...mrpRate`) সম্পূর্ণ হাতে-এন্ট্রি করা বিক্রয়-মূল্য, যার সাথে প্রকৃত কাঁচামাল-ক্রয়-মূল্যের কোনো স্বয়ংক্রিয় সংযোগ নেই। অর্থাৎ, সিস্টেমে প্রকৃত উৎপাদন-ব্যয় (Cost of Goods Manufactured) থেকে মুনাফা গণনার কোনো ব্যবস্থা নেই — সমস্ত মার্জিন রিপোর্ট (Company Earnings, usableMoney ইত্যাদি) সম্পূর্ণভাবে হাতে-নির্ধারিত রেট কার্ডের উপর নির্ভরশীল, প্রকৃত ব্যয়ের উপর নয়।

৭.২ প্রোডাকশনের পার্শ্ব-প্রতিক্রিয়া

- প্রতিটি ব্যাচ স্বয়ংক্রিয়ভাবে একটি `MaterialUsageRecord` তৈরি করে (`qty = rawKgConsumedTotal`), যাতে উৎপাদনে ব্যবহৃত কাঁচামালও "সাম্প্রতিক ব্যবহার" লগে দেখা যায়
- Stock Shortfall স্বয়ংক্রিয় সমাধান:** যদি আগে কোনো রেট কার্ড ইনভয়েসে ফিনিশড গুডস উৎপাদনের আগেই বিক্রি হয়ে থাকে (স্টক ঋণাত্মক হয়ে `StockShortfallRecord` তৈরি হয়েছিল), নতুন উৎপাদিত স্টক FIFO ভিত্তিতে (পুরনো `createdAt` আগে) সেই ঘাটতি পূরণ করে
- `deleteProductionBatch` — শর্টফল-রেজোলিউশন প্রথমে বিপরীতমুখী করে, তারপর কাঁচামাল স্টক ফেরত দেয়, তারপর ফিনিশড গুডস স্টক বিয়োগ করে

৮. স্টক নিয়ন্ত্রণ (Stock Adjustment · Stock Count · QC Hold)

পরিধি সংক্রান্ত গুরুত্বপূর্ণ নোট: এই পুরো অধ্যায়ের সব ফাংশন শুধুমাত্র `erp/products` (মূল Product তালিকা)-এর উপর কাজ করে — `purchaseMaterials` বা `finishedGoods` -এর জন্য কোনো অনুমোদন-ওয়ার্কফ্লো নেই। সেগুলোর স্টক শুধু সরাসরি এডিট (কোনো approval ছাড়া) অথবা মূল ডকুমেন্ট (Purchase/Usage/Production) মুছে/সংশোধন করেই বদলানো যায়।

৮.১ Stock Adjustment — অনুরোধ → অনুমোদন → প্রত্যাখ্যান

```
createStockAdjustmentRequest:
  quantityBefore = product.stockQty
  delta          = newQuantity - quantityBefore      (status: 'pending', স্টকে কোনো প্রভাব নেই)

approveStockAdjustment:
  nextStock = max(product.stockQty + delta, 0)        (অনুমোদনের সময়ের বর্তমান স্টকে delta প্রয়োগ হয়)
  status = getProductStatus(nextStock, minStock)

rejectStockAdjustment: কোনো স্টক-প্রভাব নেই, status = 'rejected'
```

৮.২ Stock Count — ফিজিক্যাল কাউন্ট বনাম সিস্টেম

```
systemQty = product.stockQty      (কাউন্টের মুহুর্তে স্ক্যানশট)
variance   = physicalQty - systemQty
```

এটি খাঁটি রিপোর্টিং — **কখনো `stockQty` পরিবর্তন করে না।** ভ্যারিয়েন্স পাওয়া গেলে ম্যানেজারকে আলাদাভাবে একটি Stock Adjustment তৈরি করতে হয়।

৮.৩ QC Hold — কোয়ালিটি হোল্ড রিলিজ/স্ক্র্যাপ

```
releaseQcHold:
  nextStock = product.stockQty + hold.quantity
  value = hold.quantity * hold.unitCost
  যদি value > 0: Dr Inventory (value) / Cr COGS (value) [লেজারে পোস্ট]

scrapQcHold: status = 'scrapped' - কোনো স্টক বা লেজার প্রভাব নেই
```

গ্যাপ: বর্তমান UI-তে নতুন QC Hold তৈরি করার কোনো পথ নেই — শুধু পুরনো (legacy) রেকর্ড রিলিজ/স্ক্র্যাপ করা যায়। এটি কার্যত একটি "dead feature"।

৯. রোট কার্ড / ইনভয়েসিং — মূল বিক্রয় ইঞ্জিন

এটিই সিস্টেমের প্রকৃত-জীবন্ত (UI থেকে ব্যবহৃত) বিক্রয় নথি — Company → Depot → Dealer শিপমেন্ট/চালান, যেখান থেকে একই রেকর্ড দিয়ে তিন ধরনের ভাউচার (Company/Depot/Dealer) প্রিন্ট করা যায়।

৯.১ পিস-রূপান্তর — সবচেয়ে বেশি ব্যবহৃত সূত্র

```
parsePerCtnMultiplier(perCtnBgs):  
  perCtnBgs টেক্সট (যেমন "500 ps = 1 bg") থেকে প্রথম সংখ্যাটি বের করে  
  → কোনো সংখ্যা না পোলে ডিফল্ট 1  
  
pieces(লাইন) = item.qty * parsePerCtnMultiplier(item.perCtnBgs)
```

এই `pieces` সূত্রটিই সমস্ত ইনভয়েস টোটাল, প্রতিটি ভাউচার প্রিন্ট, ড্যাশবোর্ড এবং সেলস রিপোর্টের ভিত্তি।

৯.২ computeRateCardTotals — সম্পূর্ণ ইনভয়েস গণনা

```
rawRateTotal    = Σ pieces(item) * item.rawRate  
manufRateTotal  = Σ pieces(item) * item.manufRate  
depotRateTotal  = Σ pieces(item) * item.depotRate  
dealerRateTotal = Σ pieces(item) * item.dealerRate ← ডিলারকে প্রকৃত বিলকৃত পরিমাণ  
tpRateTotal     = Σ pieces(item) * (item.tpRate ?? 0)  
mrpRateTotal    = Σ pieces(item) * (item.mrpRate ?? 0)  
  
pouchCartonAmount = manufRateTotal - rawRateTotal  
usableMoney        = depotRateTotal - manufRateTotal ← কোম্পানির মার্জিন (ডিপো পর্যন্ত)  
usableMoneyPercent = dealerRateTotal ? usableMoney / dealerRateTotal * 100 : 0  
usableUDepot       = dealerRateTotal - manufRateTotal ← কোম্পানির মার্জিন (ডিপো বাদ দিয়ে)  
usableUDepotPercent = dealerRateTotal ? usableUDepot / dealerRateTotal * 100 : 0
```

এই টোটালগুলো **denormalized** — অর্থাৎ সেভ করার সময় একবার গণনা করে সরাসরি `RateCardRecord` -এ সংরক্ষণ করা হয় (রিড করার সময় আবার গণনা হয় না)।

৯.৩ স্টক প্রভাব ও শর্টফল ট্র্যাকিং

সেভ করার সময় পুরনো ও নতুন পিস-সংখ্যার পার্থক্য (delta) `Product / FinishedGoods` -এর `stockQty` -তে প্রয়োগ হয় — **ইচ্ছাকৃতভাবে ঋণাত্মক স্টক হতে দেওয়া হয়** (উৎপাদনের আগেই বিক্রি সম্ভব)। ফিনিশড গুডস লাইনের জন্য `StockShortfallRecord` -এ ঘাটতি ট্র্যাক হয়, যা পরবর্তী প্রোডাকশন ব্যাচ দিয়ে FIFO স্বয়ংক্রিয়ভাবে সমাধান হয় (অধ্যায় ৭.২)।

গুরুত্বপূর্ণ গ্যাপ: রোট কার্ড কোনো লেজার/জার্নাল এন্ট্রি পোস্ট করে না — এটি Automatic Accounting Engine-এর সাথে সংযুক্ত নয়। এতে কোনো `paid / due` /পেমেন্ট-স্ট্যাটাসও নেই — এটি নিছক একটি ইনভয়েস/মূল্য-নির্ধারণ ডকুমেন্ট, প্রাপ্য (receivable) ট্র্যাকিং নয়।

৯.৪ DealerCategory → Sale Type → প্রাইসিং ক্যাটালগ নির্বাচন

এই নির্বাচন-লজিক প্রোভাইডারে (ব্যাকএন্ডে) নেই — এটি সম্পূর্ণভাবে রোট কার্ড UI (`app/admin/rate-card/page.tsx`)-তে ঘটে:

- "Sale type" ড্রপডাউনে জীবন্ত `DealerCategoryRecord` -সমূহ দেখানো হয়
- `isCommissionSaleType` সত্য হলে, প্রতিটি লাইনের প্রোডাক্ট-পিকার সাধারণ `Product/FinishedGoods` তালিকার বদলে **Discount Product List** দেখায়
- এমন ক্ষেত্রে, নির্বাচিত ডিসকাউন্ট প্রোডাক্টের `dealerRate` ফিল্ডটি প্রতিস্থাপিত হয়: `computeDiscountProductRates()` থেকে পাওয়া `srRate` ও `tpRate` দিয়ে — এই মানগুলো **লাইনে ফ্রিজ (হিমায়িত)** হয়ে যায়, পরবর্তীতে ডিসকাউন্ট প্রোডাক্টের কমিশন% বদলালেও পুরনো ইনভয়েস অপরিবর্তিত থাকে
- সাধারণ (non-commission) ক্ষেত্রে, রোট প্রি-ফিল হয় `ProductRecord / FinishedGoodsRecord` -এর নিজস্ব রোট কলাম থেকে (শুধু প্রথমবার নির্বাচনে — হাতে বদলানো রোট কখনো ওভাররাইট হয় না)

১০. ডিসকাউন্ট প্রোডাক্ট ও কমিশন-চেইন প্রাইসিং

অধ্যায় ৪.২ ও ৯.৪-এ বর্ণিত `DiscountProductRecord` এবং `computeDiscountProductRates` -এর পূর্ণাঙ্গ ব্যবহার-প্রবাহ:

Discount Products List (ম্যানুয়াল এন্ট্রি):

dealerRate, srCommissionPercent (ডিফল্ট ৮%), tpPercent, rawRate, manufRate, depotRate, mrpRate

লাইভ-গণনা (কখনো সংরক্ষিত হয় না):

$srRate = dealerRate * (1 + srCommissionPercent/100)$

$tpRate = srRate * (1 + tpPercent/100)$

রেট কার্ডে ব্যবহারের সময়:

RateCardLineItem.dealerRate ← srRate (হিমায়িত হয়ে সংরক্ষিত)

RateCardLineItem.tpRate ← tpRate (হিমায়িত হয়ে সংরক্ষিত)

`deleteDiscountProduct` -এ কোনো রেফারেন্স-গার্ড নেই (ক্ষতিকর নয়, কারণ রোট কার্ড লাইন রোট কপি করে রাখে, লাইভ রেফারেন্স রাখে না)।

১১. প্রোডাক্ট রিটার্ন

এটি `SalesReturnRecord` (অর্ডার-ভিত্তিক, অধ্যায় ১২.৩) থেকে সম্পূর্ণ ভিন্ন একটি এনটিটি — ডিপো বা ডিলার থেকে স্টক ফেরত নেওয়ার জন্য, কোনো নির্দিষ্ট ইনভয়েসের সাথে যুক্ত নয়।

```
computeProductReturnTotals (কার্টন-গুণক ছাড়াই, qty যেমন এন্ট্রি করা হয়েছে তেমনই):
rawRateTotal      = Σ item.qty * item.rawRate
manufRateTotal    = Σ item.qty * item.manufRate
depotRateTotal    = Σ item.qty * item.depotRate
dealerRateTotal   = Σ item.qty * item.dealerRate
tpRateTotal       = Σ item.qty * (item.tpRate ?? 0)
mrpRateTotal      = Σ item.qty * (item.mrpRate ?? 0)

companyProfit = depotRateTotal - manufRateTotal      ← একমাত্র বাস্তব মুনাফা-প্রভাব
depotProfit   = 0   (deprecated, শুধু টাইপ-সামঞ্জস্যের জন্য রাখা)
dealerProfit  = 0   (deprecated)
```

ব্যবসায়িক নিয়ম (২০২৬-০৯-১০ থেকে কার্যকর): রিটার্ন-পাঠি ডিপো হোক বা ডিলার — সবসময় `depotRate` -এ মূল্যায়িত হয়, কখনো `dealerRate` -এ নয়। এর ফলে শুধুমাত্র `companyProfit` হিসাব করলেই তা Company Earnings-এর `usableMoney` -এর বিপরীতে সঠিকভাবে নিট হয়ে যায়।

`manufacturingExpenseAmount = manufRateTotal - rawRateTotal` এবং `rawMaterialExpenseAmount = rawRateTotal × 0.30` — এই দুটি মান শুধুমাত্র প্রদর্শন/অডিটের জন্য সংরক্ষিত, ২০২৬-০৯-১০-এর পর থেকে এগুলো আর প্রকৃত `ExpenseRecord` হিসেবে পোস্ট হয় না (আগে হতো, যা দ্বৈত-গণনার সমস্যা তৈরি করত)। `createProductReturn` কোনো স্টক/লজার/এক্সপেন্স প্রভাব ফেলে না — শুধুমাত্র রেকর্ডটি সংরক্ষণ করে।

১২. সেলস অর্ডার মডিউল (বর্তমানে নিষ্ক্রিয়)

গুরুত্বপূর্ণ স্থাপত্যগত পর্যবেক্ষণ: কোডে `OrderRecord` -ভিত্তিক একটি সম্পূর্ণ, পূর্ণাঙ্গ সেলস অর্ডার সিস্টেম আছে — স্টক-ট্রাস, FEFO ব্যাচ কনজাম্পশন, সম্পূর্ণ ডাবল-এন্ট্রি লেজার পোস্টিং এবং অনুমোদন-ওয়ার্কফ্লো সহ। কিন্তু `src/app` -এর কোনো স্ক্রিন থেকে এটি ব্যবহার করা যায় না — বর্তমানে প্রকৃত বিক্রয় সম্পূর্ণভাবে Rate Card (অধ্যায় ৯) দিয়ে চলে, যেটির কোনো লেজার-সংযোগ নেই। এই অধ্যায়টি সম্পূর্ণতার জন্য নথিবদ্ধ করা হলো, ভবিষ্যতে UI যুক্ত হলে বা এই কোড অপসারণের সিদ্ধান্ত নেওয়ার সময় কাজে লাগবে।

১২.১ createOrder

```
subtotal      = Σ orderItem.unitPrice * orderItem.quantity
discount       = clamp(input.discount, 0, subtotal)
promotionalDiscount = clamp(input.promotionalDiscount, 0, subtotal - discount)
netSales       = subtotal - discount - promotionalDiscount
vat            = max(input.vat, 0)
total          = netSales + vat
paid           = clamp(input.paid, 0, total)
due            = total - paid
cogs           = Σ orderItem.purchasePrice * orderItem.quantity
paymentStatus: due===0→'paid' | paid>0→'partial' | অন্যথায়→'unpaid'
```

লেজার পোস্টিং (buildInvoiceLedgerEntries):

```
Dr Dealer (total)      Cr Sales (netSales)
Dr COGS (cogs)         Cr VAT Payable (vat)
Dr Cash (paid, যদি থাকে) Cr Inventory (cogs)
                        Cr Dealer (paid, যদি থাকে)
```

অনুমোদন-নিয়ম: `discount > 0` অথবা কোনো লাইনের `unitPrice < product.minSellingPrice` হলে `approvalStatus = 'pending'` হয়, অন্যথায় স্বয়ংক্রিয় 'approved'। স্টক FEFO ভিত্তিতে ব্যাচ থেকে কাটা হয় (`consumeBatchesFefo`)।

১২.২ updateOrder / cancelOrder / updateOrderStatus

`updateOrder` শুধু `status==='pending'` অবস্থায় সম্ভব; পুরনো লেজার এন্ট্রি বিপরীতমুখী করে নতুন এন্ট্রি পোস্ট করে (মূল এন্ট্রি কখনো মুছে ফেলা হয় না)। `cancelOrder` সম্পূর্ণ বিপরীতমুখী করে (স্টক, ব্যাচ, লেজার — সব)। `updateOrderStatus` -এ একটি বিশেষ নিয়ম আছে: স্ট্যাটাস 'completed' করলে এবং `due > 0` থাকলে, বাকি বকেয়া স্বয়ংক্রিয়ভাবে সম্পূর্ণ পরিশোধিত ধরে নেওয়া হয় ও লেজারে `Dr Cash / Cr Dealer` পোস্ট হয় — কিন্তু কোনো `CollectionRecord` তৈরি হয় না (নিচের ১২.৪-এর সাথে তুলনীয়)।

১২.৩ createSalesReturn (অর্ডার-ভিত্তিক রিটার্ন)

```
totalRefund    = Σ item.unitPrice * item.quantity (good + damaged সব আইটেম)
restockedQty   = Σ (condition==='good') item.quantity (শুধু ভালো আইটেম স্টকে ফেরত যায়)
writtenOffQty  = Σ (condition==='damaged') item.quantity
restockedCost  = Σ (good items only) purchasePrice * quantity
nextOrderDue   = max(order.due - totalRefund, 0)
```

```
লেজার: Dr Sales Return (totalRefund) / Cr Dealer (totalRefund)
        যদি restockedCost > 0: Dr Inventory (restockedCost) / Cr COGS (restockedCost)
```

১২.৪ recordCollection

```
শর্ত: amount > 0 এবং amount <= order.due
nextDue   = order.due - amount
nextPaid  = order.paid + amount
লেজার: Dr {cash/bank/mfs}(amount) / Cr Dealer(amount)
```

১৩. সেলস রিপোর্ট ও কোম্পানি আর্নিংস

১৩.১ buildCompanyEarningsSummary — কোম্পানির প্রকৃত মুনাফা

```
totalEarning = Σ(rateCard.usableMoney) - Σ(productReturn.companyProfit)
totalExpense = Σ(expense.amount) [যেসব expense.approvalStatus !== 'rejected']
netProfit = totalEarning - totalExpense

totalDealerValueSales = Σ(rateCard.dealerRateTotal) - Σ(productReturn.dealerRateTotal)
avgProfitRatioPercent = totalDealerValueSales > 0
                        ? (totalEarning / totalDealerValueSales) * 100
                        : 0
```

মাসিক ও বাৎসরিক ব্রেকডাউনও একই সূত্রে, শুধু তারিখ অনুযায়ী বাকेट করা।

১৩.২ buildSalesReportSummary — ডিলার/প্রোডাক্ট-ভিত্তিক বিক্রয়

প্রতিটি রেট কার্ডের `dealerRateTotal`-কে Sale Type অনুযায়ী তিনটি বাকটে ভাগ করে: `commission` / `others` / `unclassified`। প্রতি ডিলার ও প্রতি প্রোডাক্ট আলাদাভাবে রোল-আপ হয়।

১৩.৩ buildCategorySalesReportSummary — ক্যাটাগরিভিত্তিক বিক্রয়

একই ছয়টি রেট-টোটাল সূত্র (৯.২ দ্রষ্টব্য) প্রোডাক্ট-ক্যাটাগরি অনুযায়ী যোগ করে, তারপর প্রোডাক্ট রিটার্নের সমপরিমাণ টোটাল **বিয়োগ** করে (রিটার্ন লাইনে কার্টন-গুণক প্রযোজ্য নয়, qty সরাসরি ব্যবহৃত হয়)। এখান থেকেই `companyProfit = depotRateTotal - manufRateTotal` এবং `depotProfit = dealerRateTotal - depotRateTotal` পাওয়া যায়।

১৩.৪ buildOperationsOverview — ড্যাশবোর্ড

যেহেতু Sales Order মডিউল সচল নয়, ড্যাশবোর্ডের "টপ প্রোডাক্ট"/"টপ ডিলার" সরাসরি রেট কার্ড থেকে গণনা হয়: প্রতি প্রোডাক্টে `revenue += pieces * dealerRate`, প্রতি ডিলারে `total += card.dealerRateTotal` — শীর্ষ ৫টি করে দেখানো হয়। লো-স্টক তালিকা: `products.filter(stockQty <= minStock)`।

১৪. এক্সপেন্স ও বাজেট ব্যবস্থাপনা

১৪.১ saveExpense — লেজার পোস্টিং অনুমোদনের আগেই ঘটে

এক্সপেন্স সেভ করার সাথে সাথেই (অনুমোদনের অপেক্ষা না করে) লেজারে পোস্ট হয়ে যায় — অনুমোদন-প্রক্রিয়াটি নিছক একটি "পরবর্তী পর্যালোচনা গेट":

```
Dr resolveExpenseLedgerAccount(category) (amount)
Cr paymentMethod ('cash' বা 'bank') (amount)
```

এডিট করলে — আগের সক্রিয় লেজার-লাইন বিপরীতমুখী করে নতুন এন্ট্রি পোস্ট হয় (মূল এন্ট্রি কখনো মোছা হয় না)। `updateExpenseApproval('rejected')` — লেজার এন্ট্রি বিপরীতমুখী করে কিন্তু রেকর্ডটি অডিটের জন্য রেখে দেয়। `deleteExpense` — রেকর্ডসহ পুরো লেজার-প্রভাব সম্পূর্ণ মুছে দেয়।

১৪.২ বেতন ও ঋণ-পরিশোধ ক্যাটাগরির বিশেষ আচরণ

- ক্যাটাগরি "সেলারি": `employeeName` ট্যাগ যুক্ত হয়; `computeEmployeeSalaryTotals` নাম কেস-ইনসেনসিটিভভাবে গ্রুপ করে মোট বেতন দেখায়
- ক্যাটাগরি "ঋণ পরিশোধ": একটি এক্সপেন্স সেভ করলে **একই সাথে** একটি `LoanTransactionRecord (type: 'repayment')` -ও তৈরি/আপডেট হয় — একই টাকা একবার P&L এক্সপেন্স হিসেবে এবং একবার ঋণগ্রহীতার ব্যালেন্স-ট্রান্স হিসেবে দুইবার রেকর্ড হয়

১৪.৩ Budget — পরিকল্পনা বনাম প্রকৃত

```
getBudgetActual(expenses, budget):
  category মিলে যাওয়া (কেস-ইনসেনসিটিভ), বছর মিলে যাওয়া, এবং
  (periodType==='yearly' অথবা মাস মিলে যাওয়া) — এমন সব expense.amount-এর যোগফল

checkBudgetOverrun: actual > budgetAmount হলে →
  'warning' নোটিফিকেশন পাঠানো হয় super_admin/manager/accounts-কে
```

লক্ষণীয়: `getBudgetActual` রিজেক্টেড এক্সপেন্সও বাদ দেয় না (Company Earnings-এর হিসাব থেকে ভিন্ন, যেখানে রিজেক্টেড বাদ যায়)।

১৫. লোন ম্যানেজমেন্ট

```
computeLoanBalance:
  totalWithdrawn = Σ amount (type === 'withdrawal')
  totalRepaid    = Σ amount (type === 'repayment')
  balance = totalWithdrawn - totalRepaid    (কখনো ক্ল্যাম্প করা হয় না – বেশি পরিশোধ হলে ঋণাত্মকও হতে পারে)
```

```
computeLoanMonthlySchedule (মাসভিত্তিক তালিকা, প্রথম লেনদেন থেকে চলতি মাস পর্যন্ত):
  opening  = আগের মাসের closing
  closing  = opening + এই-মাসের-withdrawals - এই-মাসের-repayments
  → পরবর্তী মাসের opening হিসেবে বহন হয় (বছর পরিবর্তনেও)
```

`saveLoanTransaction` সরাসরি কোনো লেজার-প্রভাব ফেলে না — শুধুমাত্র "ঋণ পরিশোধ" ক্যাটাগরির এক্সপেন্স দিয়ে পরোক্ষভাবে লেজারে পৌঁছায় (অধ্যায় ১৪.২)। `deleteLoanAccount` — লেনদেনের ইতিহাস থাকলে ব্লক হয়।

১৬. ক্যাশ মেইনটেন্যান্স ও ইনভেস্টর

১৬.১ CashMaintenanceRecord — সমান্তরাল, non-P&L ক্যাশ লগ

এটি ইচ্ছাকৃতভাবে Automatic Accounting Engine/ `ledgerEntries` -এর বাইরে রাখা হয়েছে, যাতে এটি Company Earnings-এর নিট মুনাফার হিসাবে দ্বৈত-গণনা তৈরি না করে। ক্যাটাগরি: ঋণ পরিশোধ, নতুন মার্কেট ইনভেস্টমেন্ট, পণ্য/প্যাকেজিং ক্রয়, ডিপো কমিশন, ডিলার পরিবহন খরচ।

যেসব ফাংশন এই একই কালেকশনে লেখে (নিজস্ব ফর্ম ছাড়াও):

- `buildPurchaseCashEntries` — Purchase/VendorPayment-এর কাঁচামাল-বনাম-প্যাকেজিং অনুপাত-স্প্লিট (অধ্যায় ৬.২)
- `saveInvestor` — প্রতিটি ইনভেস্টরের জন্য "নতুন মার্কেট ইনভেস্টমেন্ট" ক্যাটাগরিতে একটি লিংকড এন্ট্রি রাখে

১৬.২ InvestorRecord

`amount` একটি ক্রমপুঞ্জিত (running) মোট — নতুন লেনদেন হিসেবে নয়, বরং প্রতিবার এডিট করলে লিংকড `CashMaintenanceRecord` -এর `amount` -ও সরাসরি ওভাররাইট হয়।

গ্যাপ: ইনভেস্টরের বিনিয়োগ কখনো Chart of Accounts-এর "Share Capital" অ্যাকাউন্টে পোস্ট হয় না — Balance Sheet-এর Equity অংশে দেখাতে হলে আলাদা ম্যানুয়াল Journal Entry প্রয়োজন।

১৭. অ্যাকাউন্টিং — চার্ট অফ অ্যাকাউন্টস, জার্নাল, ব্যাংক, লেজার, ব্যালান্স শিট

১৭.১ দুটি পোস্টিং পদ্ধতি, একটি লেজার

সিস্টেমে দুই ধরনের অ্যাকাউন্টিং-পোস্টিং একসাথে চলে, দুটোই শেষমেশ একই `LedgerEntries` কালেকশনে যায়:

- স্বয়ংক্রিয় অ্যাকাউন্টিং ইঞ্জিন** — বিক্রয়, কালেকশন, এক্সপেন্স, প্রোডাক্ট রিটার্ন, কমিশন পেআউট, ব্যাংক লেনদেন — এসব সরাসরি নির্দিষ্ট `LedgerAccount` কী (যেমন `'sales'`, `'cogs'`, `'dealer'`) দিয়ে পোস্ট করে
- ম্যানুয়াল জার্নাল এন্ট্রি** — `account:'manual' + accountRef` (নির্দিষ্ট Chart of Accounts আইডি) — যেকোনো ঘটনার জন্য (মালিকের মূলধন, ব্যাংক ঋণ, অবচয় ইত্যাদি)

```
resolveLedgerAccountRecord(chartOfAccounts, entry):  
  account === 'manual' → chartOfAccounts[entry.accountRef]  
  অন্যথায় → যে অ্যাকাউন্টের ledgerAccount === entry.account
```

১৭.২ Chart of Accounts

এটি গ্রুপ — `Asset/Liability/Equity/Revenue/Expense`। `isSystem:true` অ্যাকাউন্টের `code / type` কখনো বদলানো যায় না (শুধু নাম/ব্যালেন্স/স্ট্যাটাস বদলানো যায়)। `deleteChartOfAccount` ব্লক হয় যদি সিস্টেম-অ্যাকাউন্ট হয় অথবা কোনো পোস্টেড জার্নাল-লাইন তা রেফারেন্স করে। `seedStandardChartOfAccounts` — `code` অনুযায়ী idempotent (একই কোড দুইবার যোগ হয় না); স্ট্যান্ডার্ড চার্টে ১০টি Asset, ৪টি Liability, ২টি Equity, ৩টি Revenue, ১৮টি Expense কোড আছে।

১৭.৩ Journal Entry — একমাত্র লিখন-সময়ে যাচাইকৃত ডাবল-এন্ট্রি

```
শর্ত: কমপক্ষে ২টি লাইনে অর্থ থাকতে হবে  
প্রতি লাইনে: debit এবং credit একসাথে ধনাত্মক থাকতে পারবে না; ঋণাত্মক নিষিদ্ধ  
totalDebit = Σ line.debit  
totalCredit = Σ line.credit  
শর্ত: |totalDebit - totalCredit| <= 0.01 (নাহলে সেভই হবে না)
```

এটা একমাত্র জায়গা যেখানে ডেবিট=ক্রেডিট সরাসরি যাচাই (validate) করা হয় — বাকি সব "স্বয়ংক্রিয়" পোস্টার তাদের বিল্ট-ইন গঠন অনুযায়ীই সবসময় ব্যালেন্সড থাকে। `reverseJournalEntry` — কখনো হার্ড-ডিলিট নয়; `status:'reversed'` করে সমান-বিপরীত এন্ট্রি পোস্ট করে।

১৭.৪ ব্যাংক অ্যাকাউন্ট ও লেনদেন

প্রতিটি `BankAccountRecord` তৈরির সাথে সাথেই স্বয়ংক্রিয়ভাবে একটি লিংকড `ChartOfAccountRecord` (type: asset, isSystem: true) তৈরি হয় — এভাবেই ব্যাংক ব্যালেন্স জেনারেল লেজারে বিশেষ কোনো কোড ছাড়াই প্রতিফলিত হয়।

লেনদেনের ধরন	ডেবিট	ক্রেডিট
deposit (জমা)	ব্যাংক অ্যাকাউন্ট	Cash
withdrawal (উত্তোলন)	Cash	ব্যাংক অ্যাকাউন্ট
transfer (স্থানান্তর)	গন্তব্য ব্যাংক অ্যাকাউন্ট	উৎস ব্যাংক অ্যাকাউন্ট
bank_charge (ব্যাংক চার্জ)	Bank Charge (এক্সপেন্স)	ব্যাংক অ্যাকাউন্ট
cheque_issued (চেক ইস্যু)	কাউন্টার অ্যাকাউন্ট	ব্যাংক অ্যাকাউন্ট
cheque_deposited (চেক জমা)	ব্যাংক অ্যাকাউন্ট	কাউন্টার অ্যাকাউন্ট

`deleteBankAccount` — কোনো লেনদেন-ইতিহাস থাকলে ব্লক হয় (শুধু নিষ্ক্রিয় করা যায়)।

১৭.৫ General Ledger → Trial Balance → Balance Sheet (তিন-স্তরের ডেরিভেশন)

```
buildGeneralLedger (প্রতিটি অ্যাকাউন্টের জন্য):  
  running = account.openingBalance  
  প্রতিটি লেনদেন তারিখ-ক্রমানুসারে:  
    running += entry.debit - entry.credit  
  account.closingBalance = running (চূড়ান্ত running)
```

```
buildTrialBalance:  
  closingBalance > 0 → Debit কলামে  
  closingBalance < 0 → Credit কলামে (ধনাত্মক করে)  
  totalDebit = totalCredit (কাঠামোগতভাবে সবসময় মিলে যায়, যেহেতু প্রতিটি পোস্টিং নিজেই ব্যালেন্সড জোড়া)
```

```
buildBalanceSheet:  
  assets = Σ closingBalance [type==='asset']  
  liabilities = Σ -closingBalance [type==='liability']  
  equity = Σ -closingBalance [type==='equity']  
  revenue = Σ -closingBalance [type==='revenue']  
  expense = Σ closingBalance [type==='expense']  
  currentPeriodNetProfit = revenue - expense  
  totalEquity = equityAccounts + currentPeriodNetProfit (বছর-শেষ ক্লোজিং ছাড়াই লাইভ)  
  totalLiabilitiesAndEquity = totalLiabilities + totalEquity  
  isBalanced = |totalAssets - totalLiabilitiesAndEquity| < 1
```

১৭.৬ কোন মডিউল লেজারে কী পোস্ট করে — সম্পূর্ণ তালিকা

মডিউল/ফাংশন	ডেবিট	ক্রেডিট
createOrder / updateOrder (নিষ্ক্রিয়)	Dealer(total), COGS(cogs), Cash(paid)	Sales(netSales), VAT(vat), Inventory(cogs), Dealer(paid)
recordCollection (নিষ্ক্রিয়)	Cash/Bank/MFS(amount)	Dealer(amount)
updateOrderStatus → completed (নিষ্ক্রিয়)	Cash(due)	Dealer(due)
createSalesReturn (নিষ্ক্রিয়)	Sales Return(refund), Inventory(restockedCost)	Dealer(refund), COGS(restockedCost)
releaseQcHold	Inventory(value)	COGS(value)
saveExpense	ক্যাটাগরি-ম্যাপড এক্সপেন্স অ্যাকাউন্ট	Cash অথবা Bank
recordCommissionPayout	Commission(amount)	Cash অথবা Bank
createJournalEntry	ব্যবহারকারী-নির্ধারিত	ব্যবহারকারী-নির্ধারিত
recordBankTransaction	অধ্যায় ১৭.৪-এর টেবিল দ্রষ্টব্য	

লক্ষণীয় অনুপস্থিতি: `createPurchase` ও `recordVendorPayment` কখনো `LedgerEntries`-এ লেখে না — ফলে "Accounts Payable"/"Supplier" অ্যাকাউন্ট বাস্তব ক্রয়-কার্যক্রম থেকে কখনো স্বয়ংক্রিয়ভাবে পূর্ণ হয় না।

১৮. কমিশন রুল ও সেলস টার্গেট (শুধু কাঠামো, বাস্তবায়িত নয়)

গ্যাপ: নিচের দুটি মডিউলের সম্পূর্ণ ডেটা-মডেল ও ব্যাকএন্ড CRUD ফাংশন কোডে আছে, কিন্তু `src/app`-এ কোনো UI স্ক্রিন নেই — `defaultData.ts`-এ উভয়ই খালি সিড করা হয়।

১৮.১ Commission Rule & Payout

`CommissionRuleRecord` দুই ধরনের হতে পারে — `'slab'` (বিক্রয়ের পরিমাণ-ভিত্তিক ধাপ, যেমন "১০ লাখের উপরে ৫%") অথবা `'product'` (প্রোডাক্টভিত্তিক %)। কিন্তু `recordCommissionPayout` ফাংশনটি নিজে কোনো "রুল × বিক্রয়" গণনা করে না — এটি কেবল আগে থেকে গণনা করা `salesAmount / commissionAmount` গ্রহণ করে রেকর্ড করে এবং লেজারে পোস্ট করে। যেহেতু এই ইনপুট সরবরাহ করার মতো কোনো UI নেই, তাই প্রকৃত কমিশন-গণনা লজিকটি **স্ক্রিমা আকারে বিদ্যমান কিন্তু বাস্তবায়িত নয়**।

১৮.২ Sales Target

`SalesTargetRecord`-এ শুধু `targetAmount` সংরক্ষিত হয়; "অর্জন" (Achievement) ডিজাইন অনুযায়ী লাইভ গণনা হওয়ার কথা (Budget Actual-এর মতো), কিন্তু কোডে এমন কোনো ফাংশন (`getSalesTargetAchievement` জাতীয়) পাওয়া যায়নি। এটিও স্ক্রিমা-শুধু।

১৯. ড্যাশবোর্ড ও রিপোর্টস হাব

১৯.১ Dashboard

`buildOperationsOverview` ও `buildCompanyEarningsSummary` -এর ফলাফল KPI কার্ড, ৬-মাসের বার-চার্ট, সাম্প্রতিক রোট কার্ড/এক্সপেন্স তালিকা এবং টপ ডিলার/প্রোডাক্ট তালিকায় প্রদর্শিত হয়। এখানে নতুন কোনো ব্যবসায়িক গণনা নেই — শুধু উপস্থাপনা ও একটি প্রসাধনী ফল্ট-সাইজ সমন্বয় ফাংশন।

১৯.২ Reports Hub

একই জায়গা থেকে Sales/Expense/Purchase/Vendor/Loan — পাঁচটি রিপোর্ট ট্যাব চালানো যায়, প্রতিটিতে তারিখ-রেঞ্জ ফিল্টার এবং এক্সপোর্ট/প্রিন্ট সুবিধা:

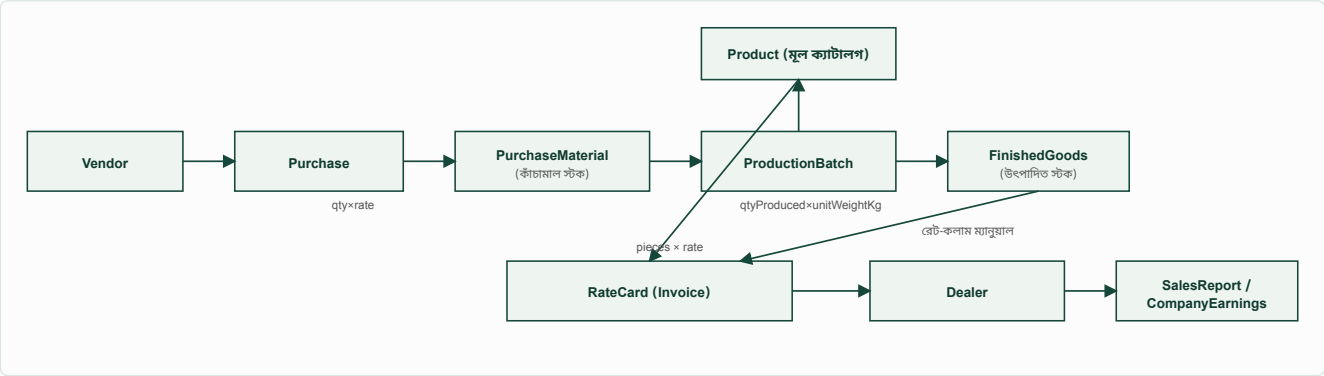
ট্যাব	মূল হিসাব
Expense	ফিল্টার করা এক্সপেন্সের যোগফল ($\sum$ amount)
Purchase	ফিল্টার করা ক্রয়ের যোগফল ($\sum$ totalAmount)
Vendor	প্রতি ভেন্ডরে <code>computeVendorDue</code> , শুধু ফিল্টার করা ভেন্ডরদের যোগফল
Loan	ফিল্টার করা লেনদেনের withdrawal-যোগফল ও repayment-যোগফল আলাদাভাবে

১৯.৩ অন্যান্য স্ক্রিন-স্তরের যুক্তি

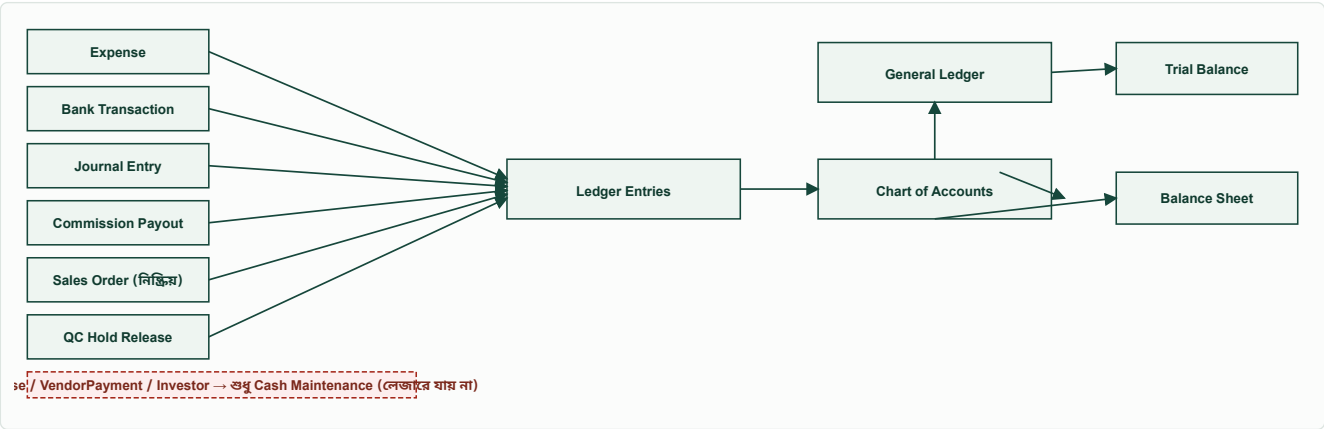
- StockControlSection:** ব্যাচগুলো FEFO অনুযায়ী (আগে মেয়াদ শেষ হওয়া আগে) সাজানো হয়; `daysUntil = ceil((expiryDate-এখন)/86400000)` দিয়ে "কত দিন বাকি" ব্যাজ
- StockOverviewScreen:** নতুন প্রোডাক্টের জন্য SKU স্বয়ংক্রিয় তৈরি (`generateSku`), স্টার্টার-ক্যাটালগ থেকে বান্ড আমদানি
- AccountingScreen (UI-স্তর যাচাই):** জার্নাল ফর্মের লাইভ ব্যালেন্স-চেক (`|totalDebit-totalCredit| < 0.01`) সেভ বাটন সক্রিয়/নিষ্ক্রিয় করে; ট্রায়াল ব্যালান্স ও ব্যালান্স শিটে অনুরূপ "Balanced" ব্যাজ (টলারেন্স ১)

২০. আন্তঃমডিউল ডেটা-ফ্লো (সম্পূর্ণ ছবি)

২০.১ পণ্য ও মূল্য-প্রবাহ (Goods & Value Flow)



২০.২ অ্যাকাউন্টিং প্রবাহ (Accounting / Ledger Flow)



লাল ডায়মন্ড বক্সটি (Purchase/VendorPayment/Investor) ইচ্ছাকৃতভাবে সাধারণ লেজার-প্রবাহের বাইরে — এগুলো শুধুমাত্র Cash Maintenance-এ যায়, যা অধ্যায় ৬ ও ১৬-এ ব্যাখ্যা করা হয়েছে।

২১. গণনা-সূত্রের মাস্টার রেফারেন্স টেবিল

মান	সূত্র	ফাইল
পণ্য স্ট্যাটাস	stockQty<=0→out-of-stock; <=minStock→low-stock; নয়তো active	utils.ts
SR/TP রেট (ডিসকাউন্ট প্রোডাক্ট)	srRate=dealerRate×(1+sr%/100); tpRate=srRate×(1+tp%/100)	utils.ts
রেট কার্ড পিস	qty × parsePerCtnMultiplier(perCtnBgs)	utils.ts
রেট কার্ড usableMoney	depotRateTotal – manufRateTotal	provider.tsx
রেট কার্ড usableUDepot	dealerRateTotal – manufRateTotal	provider.tsx
ভেন্ডর বকেয়া	Σ purchase.due (লাইভ)	utils.ts
ডিলার বকেয়া	Σ order.due, অ-বাতিল (লাইভ)	utils.ts
লোন ব্যালেন্স	Σ withdrawal – Σ repayment (লাইভ)	utils.ts
প্রোডাকশন কঁচামাল-খরচ	qtyProduced × unitWeightKg	provider.tsx
প্রোডাক্ট রিটার্ন companyProfit	depotRateTotal – manufRateTotal (সবসময় depotRate-এ)	provider.tsx
কোম্পানি আর্বিংস নেট প্রফিট	Σ usableMoney – Σ productReturn.companyProfit – Σ expense.amount	utils.ts
গড় প্রফিট রেশিও	totalEarning / totalDealerValueSales × 100	utils.ts
বাজেট প্রকৃত (Actual)	Σ expense.amount, একই ক্যাটাগরি+পিরিয়ড (লাইভ)	provider.tsx
জেনারেল লেজার ক্লোজিং	openingBalance + Σ(debit–credit), তারিখক্রমে	utils.ts
ট্রায়াল ব্যালান্স	closingBalance>0→Debit; <0→Credit (ধনাত্মক করে)	utils.ts
ব্যালান্স শিট নিট মুনাফা	Σ(–revenue closing) – Σ(expense closing)	utils.ts
ব্যালান্স শিট isBalanced	Assets – (Liabilities+Equity) < 1	utils.ts
জার্নাল ব্যালেন্স যাচাই	ΣDebit – ΣCredit <= 0.01 (সেভ-টাইম শর্ত)	provider.tsx

২২. সিস্টেমের সীমাবদ্ধতা ও গ্যাপ (সারসংক্ষেপ)

নিচে সম্পূর্ণ কোডবেস পর্যালোচনায় শনাক্ত হওয়া সব গুরুত্বপূর্ণ স্থাপত্যগত সীমাবদ্ধতা একত্রে দেওয়া হলো — এগুলো বাগ নয়, বরং ডিজাইন-সিদ্ধান্ত বা অসম্পূর্ণ বাস্তবায়ন, যা ভবিষ্যৎ পরিকল্পনার জন্য গুরুত্বপূর্ণ:

- কঁচামালের কোনো ব্যয়/গড়-ব্যয় ট্র্যাকিং নেই — `PurchaseMaterialRecord`-এ cost ফিল্ড নেই।
- রেসিপি-কন্সটিং নেই — উৎপাদন শুধু ওজন/পরিমাণ রূপান্তর করে, ফিনিশড গুডসের মূল্য প্রকৃত কঁচামাল-ব্যয় থেকে গণনা হয় না; সব রেট হাতে-এন্ট্রি।
- Purchase ও VendorPayment লেজারে পোস্ট হয় না — Accounts Payable/Supplier অ্যাকাউন্ট স্বয়ংক্রিয়ভাবে কখনো আপডেট হয় না।
- ইনভেন্টের ইকুইটি লেজারে পোস্ট হয় না — Balance Sheet-এ দেখাতে ম্যানুয়াল Journal Entry প্রয়োজন।
- Stock Adjustment/Count শুধু মূল Product তালিকার জন্য — কঁচামাল/ফিনিশড গুডসের জন্য কোনো অনুমোদন-ওয়ার্কফ্লো নেই।
- QC Hold কার্যত নিষ্ক্রিয় — তৈরির কোনো পথ নেই, শুধু পুরনো রেকর্ড রিলিজ/স্ক্যাপ করা যায়।
- Commission Rule ও Sales Target স্কিমা-শুধু — কোনো UI স্ক্রিন নেই, প্রকৃত গণনা-লজিক বাস্তবায়িত নয়।
- Sales Order মডিউল সম্পূর্ণ কিন্তু নিষ্ক্রিয় — প্রকৃত বিক্রয় Rate Card দিয়ে চলে, যার কোনো লেজার-সংযোগ নেই।
- DealerCategory ডিলিটে কোনো রেফারেন্স-চেক নেই — ব্যবহৃত ক্যাটাগরি ডিলিট হলে সংশ্লিষ্ট রেকর্ড "Unclassified" হয়ে যায়।
- Rate Card-এ কোনো paid/due/payment-status নেই — এটি নিছক একটি ইনভয়েস, প্রাপ্য-ট্র্যাকিং নয়।
- Firebase RTDB-তে সার্ভার-সাইড ট্রানজ্যাকশন লক নেই — client-computed delta পদ্ধতিতে concurrent-write race condition তাত্ত্বিকভাবে সম্ভব।

২৩. অ-কার্যকরী প্রয়োজনীয়তা

ক্ষেত্র	বিবরণ
ডেটা সঙ্গতি	"Derive, Don't Store" নীতি — কোনো ব্যালান্স ক্যাশ করা হয় না, প্রতিবার মূল লেনদেন থেকে লাইভ গণনা হয় (অধ্যায় ১.৪)
পারফরম্যান্স	ডেটার আকার বাড়ার সাথে সাথে প্রতিটি রিপোর্ট/লেজার পুরো ডেটাসেট থেকে পুনরায় গণনা হওয়ায় ভবিষ্যতে ক্লায়েন্ট-সাইড পারফরম্যান্স বিবেচ্য বিষয় হতে পারে
অডিট ট্রেইল	প্রতিটি গুরুত্বপূর্ণ অপারেশন <code>writeActivity()</code> দিয়ে লগ হয় — কে, কী মডিউল, পুরনো/নতুন মান
বিজ্ঞপ্তি	লো-স্টক, বাজেট ওভাররান, স্টক অ্যাডজাস্টমেন্ট অনুরোধ ইত্যাদির জন্য নির্দিষ্ট রোলে ওয়ার্নিং নোটিফিকেশন
প্রিন্ট/এক্সপোর্ট	প্রতিটি প্রধান স্ক্রিনে ক্লায়েন্ট-সাইড HTML ভাউচার প্রিন্ট এবং CSV/Excel/PDF এক্সপোর্ট; PDF এক্সপোর্টে বাংলা ইউনিকোড ফন্ট (Noto Sans Bengali) এমবেড করা হয় যাতে বাংলা টেক্সট সঠিকভাবে দেখা যায়
অনুমতি নিয়ন্ত্রণ	রোল-ভিত্তিক <code>module:action</code> পারমিশন প্রতিটি সংবেদনশীল স্ক্রিন/বাটনে প্রয়োগ করা হয়
রিভার্সাল নীতি	কোনো আর্থিক এন্ট্রি সরাসরি মোছা হয় না (এক্সপেন্স ডিলিট বাদে) — সবসময় "বিপরীত এন্ট্রি পোস্ট করে" রিভার্স করা হয়, যাতে সম্পূর্ণ অডিট-ইতিহাস বজায় থাকে

উপসংহার: Recipe Food ERP একটি বাস্তবিক, ব্যবহারযোগ্য মাল্টি-মডিউল সিস্টেম, যেখানে ক্রয়-উৎপাদন-বিক্রয়-অ্যাকাউন্টিং-এর মূল প্রবাহ শক্তভাবে সংযুক্ত এবং প্রতিটি সংখ্যা সুনির্দিষ্ট সূত্র মেনে গণনা হয়। তবে উপরে চিহ্নিত গ্যাপগুলো (বিশেষত রেসিপি-কন্সটিং, Accounts Payable পোস্টিং, এবং কমিশন/টার্গেট মডিউল) ভবিষ্যতের উন্নয়ন-পরিকল্পনায় অগ্রাধিকার পাওয়া উচিত।